

Part A - Basic grep and Filtering

1. Display all lines containing the word failed.

```
grep "failed" security_log.txt
```

2. Display all lines containing failed, ignoring case.

```
grep -i "failed" security_log.txt
```

3. Display all lines that contain either Failed or Denied.

```
grep -E "Failed|Denied" security_log.txt
```

4. Display all lines that do not contain the word INFO.

```
grep -v "INFO" security_log.txt
```

5. Display all lines containing the exact word root.

```
grep -w "root" security_log.txt
```

6. Display all occurrences of the word attack along with their line numbers.

```
grep -n "attack" security_log.txt
```

7. Count the number of lines containing ERROR.

```
grep -c "ERROR" security_log.txt
```

8. Display only the filenames of files whose contents contain the word malware, assuming multiple log files are available.

```
grep -l "malware" *.txt
```

9. Display the first five lines containing Suspicious.

```
grep "Suspicious" security_log.txt | head -5
```

10. Display the last five failed-login entries.

```
grep -i "Failed login" security_log.txt | tail -5
```

Part B - Regular Expression Extraction

11. Extract all IPv4 addresses from the log.

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' security_log.txt
```

12. Extract all IP addresses belonging to the 10.x.x.x private network.

```
grep -Eo '\b10\.([0-9]{1,3}\.){2}[0-9]{1,3}\b' security_log.txt
```

13. Extract all addresses belonging to the 192.168.x.x network.

```
grep -Eo '\b192\.168\.([0-9]{1,3}\.){0-1}[0-9]{1,3}\b' security_log.txt
```

14. Extract all email addresses.

```
grep -Eo '[:alnum:]_%-]+@[[:alnum:]_-]+\.[[:alpha:]]+' security_log.txt
```

15. Extract only email addresses whose domain ends in .com.

```
grep -Eo '[:alnum:]_%-]+@[[:alnum:]_-]+\.[com]\b' security_log.txt
```

16. Extract email addresses whose username contains at least one digit.

```
grep -Eo '[:alnum:]_%-]*[0-9][[:alnum:]_%-]*@[[:alnum:]_-]'
```

261100690009 – Anshul A Salian

```
] + \. [[:alpha:]]+ ' security_log.txt
```

17. Extract all URLs beginning with http:// or https://.

```
grep -Eo 'https?://[[:space:]]+' security_log.txt
```

18. Extract only HTTPS URLs.

```
grep -Eo 'https://[[:space:]]+' security_log.txt
```

19. Extract URLs ending with .php.

```
grep -Eo 'https?://[[:space:]]+\.php\b' security_log.txt
```

20. Extract all CVE identifiers in the form: CVE-YYYY-NNNN

```
grep -Eo 'CVE-[0-9]{4}-[0-9]{4,}' security_log.txt
```

21. Extract only CVEs reported for the year 2025.

```
grep -Eo 'CVE-2025-[0-9]{4,}' security_log.txt
```

22. Extract all 32-character hexadecimal MD5 hashes.

```
grep -Eo '\b[[:xdigit:]]{32}\b' security_log.txt
```

23. Extract all 40-character hexadecimal SHA-1 hashes.

```
grep -Eo '\b[[:xdigit:]]{40}\b' security_log.txt
```

24. Extract all 64-character hexadecimal SHA-256 hashes.

```
grep -Eo '\b[[:xdigit:]]{64}\b' security_log.txt
```

25. Extract all timestamps having the format: YYYY-MM-DD HH:MM:SS

```
grep -Eo '[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2}' security_log.txt
```

Part C - Anchors and Regex Patterns

26. Display all lines beginning with a timestamp.

```
grep -E '^\[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2}\[' security_log.txt
```

27. Display all lines beginning with the word Suspicious.

```
grep '^Suspicious' security_log.txt
```

28. Display lines ending with a numeric value.

```
grep -E '[0-9]+$' security_log.txt
```

29. Display lines beginning with ERROR or WARNING.

```
grep -E '^(ERROR|WARNING)' security_log.txt
```

30. Display lines whose first character is a digit.

```
grep -E '^'[0-9] security_log.txt
```

31. Find lines ending in .exe.

```
grep -E '\.exe$' security_log.txt
```

261100690009 – Anshul A Salian

32. Find lines containing filenames ending in .exe, .sh, or .pdf.

```
grep -E '[:,alnum:]_.-]+\.(exe|sh|pdf)\b' security_log.txt
```

33. Identify lines containing the exact word root, but not words such as rooted.

```
grep -w "root" security_log.txt
```

34. Display blank lines in the file.

```
grep '^$' security_log.txt
```

35. Display all nonblank lines.

```
grep -v '^$' security_log.txt
```

Part D - Linux Filters and Pipelines

36. Extract all IPv4 addresses, sort them, and remove duplicates.

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' security_log.txt | sort -V | uniq
```

37. Count the number of unique IPv4 addresses present in the log.

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' security_log.txt | sort -V | uniq | wc -l
```

38. Extract all IP addresses and display them in sorted order.

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' security_log.txt | sort -V
```

39. Determine which IP addresses appear more than once.

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' security_log.txt | sort -V | uniq -c | awk '$1 > 1'
```

261100690009 – Anshul A Salian

40. Determine the IP address that occurs most frequently.

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' security_log.txt | sort | uniq -c | sort -nr | head -1
```

41. Extract all usernames from authentication entries and display unique usernames.

```
grep -i "login for user" security_log.txt | grep -Eo 'user [[:alnum:]]_+' | awk '{print $2}' | sort -u
```

42. Count the number of authentication attempts associated with each username.

```
grep -i "login for user" security_log.txt | grep -Eo 'user [[:alnum:]]_+' | awk '{print $2}' | sort | uniq -c
```

43. Extract all HTTP status codes and determine how many times each status code occurs.

```
grep -Eo 'status [0-9]{3}' security_log.txt | awk '{print $2}' | sort | uniq -c
```

44. Display the five most frequently occurring IP addresses.

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' security_log.txt | sort | uniq -c | sort -nr | head -5
```

45. Display the three most frequently occurring usernames.

```
grep -i "login for user" security_log.txt | grep -Eo 'user [[:alnum:]]_+' | awk '{print $2}' | sort | uniq -c | sort -nr | head -3
```

46. Extract all destination ports and display them in numerical order.

```
grep -Eo 'port [0-9]+' security_log.txt | awk '{print $2}' | sort -n
```

261100690009 – Anshul A Salian

47. Remove duplicate port numbers.

```
grep -Eo 'port [0-9]+' security_log.txt | awk '{print $2}' | sort -n | uniq
```

48. Count the number of unique destination ports.

```
grep -Eo 'port [0-9]+' security_log.txt | awk '{print $2}' | sort -n | uniq | wc -l
```

49. Extract all file extensions appearing in the log and determine their frequency.

```
grep -Eo '\.[:,alnum:]]+' security_log.txt | sort | uniq -c | sort -nr
```

50. Find the five most frequently occurring domains.

```
grep -Eo '\b([[:alnum:]]-]+\.\.)+[[:alpha:]]{2,}\b' security_log.txt | sort | uniq -c | sort -nr | head -5
```

Part E - Combining grep Conditions

51. Display lines containing both an IP address and the word Failed.

```
grep -E 'Failed' security_log.txt | grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}'
```

52. Display lines containing both root and Failed.

```
grep -i "Failed" security_log.txt | grep -w "root"
```

53. Display lines containing either download or upload.

```
grep -Ei 'download|upload' security_log.txt
```

54. Display suspicious entries containing either .exe or .sh.

```
grep "Suspicious" security_log.txt | grep -E '\.(exe|sh)\b'
```

55. Display lines containing a CVE identifier and an IP address.

```
grep -E 'CVE-[0-9]{4}-[0-9]{4,}' security_log.txt | grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}'
```

56. Display lines containing both a filename and a hash value.

```
grep -E '\.(exe|sh|pdf)\b' security_log.txt | grep -Ei 'MD5=|SHA1|SHA256'
```

57. Display entries containing an IP address followed by a port.

```
grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}: [0-9]+|([0-9]{1,3}\.){3}[0-9]{1,3}.*port [0-9]+' security_log.txt
```

58. Display HTTP requests containing both a URL and status code 404.

```
grep -E 'https?:/*.*status 404' security_log.txt
```

59. Display authentication failures but exclude entries generated from 127.0.0.1.

```
grep -i "Failed login" security_log.txt | grep -v "127\.0\.0\.1"
```

60. Display attack-related entries but exclude informational (INFO) messages.

```
grep -Ei 'attack|exploit|malware|suspicious' security_log.txt | grep -v "INFO"
```

Part F - Negative Matching

61. Extract all IPv4 addresses except those beginning with 127..

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' security_log.txt | grep -v '^127\.'
```


261100690009 – Anshul A Salian

62. Display all log entries except successful login events.

```
grep -Eiv 'successful login' security_log.txt
```

63. Display all entries that do not contain INFO, DEBUG, or SUCCESS.

```
grep -Ev 'INFO|DEBUG|SUCCESS' security_log.txt
```

64. Find HTTP requests whose status code is not 200.

```
grep -E 'https?:/' security_log.txt | grep -Ev 'status 200'
```

65. Display suspicious traffic excluding connections to 192.168.x.x.

```
grep "Suspicious" security_log.txt | grep -Ev '192\.168\.'
```

66. Extract email addresses except those belonging to the organization's domain.

```
grep -Eo '[[[:alnum:]] ._%+-]+@[[:alnum:]] .-]+\.[[:alpha:]]+' security_log.txt |  
grep -v '@example\.com$'
```

Part G - Redirection & Advanced Pipelines

67. Extract all IPv4 addresses and save them into: ips.txt

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' security_log.txt > ips.txt
```

68. Extract all email addresses and save them into: emails.txt

```
grep -Eo '[[[:alnum:]] ._%+-]+@[[:alnum:]] .-]+\.[[:alpha:]]+' security_log.txt >  
emails.txt
```

69. Extract all URLs and save them into: urls.txt

```
grep -Eo 'https?:/[^[[:space:]]+' security_log.txt > urls.txt
```

70. Extract all hashes and save them into: hashes.txt

```
grep -Eio '\b[[:xdigit:]]{32}|\b[[:xdigit:]]{40}|\b[[:xdigit:]]{64}'  
security_log.txt > hashes.txt
```

71. Extract failed-login entries and save them into: failed_login.txt

```
grep -i "Failed login" security_log.txt > failed_login.txt
```

72. Extract suspicious entries and append them to: incident_report.txt

```
grep "Suspicious" security_log.txt >> incident_report.txt
```

73. Store unique IP addresses in: unique_ips.txt

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' security_log.txt | sort -V | uniq >  
unique_ips.txt
```

74. Store IP-frequency statistics in: ip_frequency.txt

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' security_log.txt | sort | uniq -c | sort -  
nr > ip_frequency.txt
```

75. Redirect normal output to results.txt and errors to errors.txt.

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' security_log.txt > results.txt 2>  
errors.txt
```

76. Extract all domains from the log entries containing Domain accessed, convert the domain names to lowercase, sort them alphabetically, remove duplicates, and save the result in domains.txt.

```
grep "Domain accessed" security_log.txt | grep -Eo '([[:alnum:]]-  
[+\.])+[[:alpha:]]{2,}' | tr '[:upper:]' '[:lower:]' | sort -u > domains.txt
```

77. Identify all IP addresses involved in failed-login attempts. Convert the output into a clean list containing one unique IP per line and save it as failed_login_ips.txt.

```
grep -i "Failed login" security_log.txt | grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}'  
| sort -V -u > failed_login_ips.txt
```

Combining Filters & Log Transformation

78. Extract all failed-login usernames and save them into: failed_users.txt

```
grep -i "Failed login" security_log.txt | grep -Eo 'user [[:alnum:]]_+' | awk  
'{print $2}' > failed_users.txt
```

79. Extract all domains and save them in lowercase into: domains.txt

```
grep -Eo '([[:alnum:]]-|\.)+[[:alpha:]]{2,}' security_log.txt | tr '[:upper:]'  
'[:lower:]' | sort -u > domains.txt
```

80. Extract all MD5 hashes, convert them to uppercase, and save them into: md5_hashes.txt

```
grep -Eo '([[:xdigit:]]{32})' security_log.txt | tr '[:lower:]' '[:upper:]' >  
md5_hashes.txt
```

81. Extract source IP addresses from failed-login records, sort them, remove duplicates, and redirect the result to: failed_ips.txt

```
grep -i "Failed login" security_log.txt | grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}'  
| sort -V -u > failed_ips.txt
```

82. Create a file named clean_log.txt in which all square brackets are removed from the original log.

```
sed 's/\[//g; s/\]//g' security_log.txt > clean_log.txt
```

83. Create normalized_log.txt by converting the complete log file to lowercase.

```
tr '[:upper:]' '[:lower:]' < security_log.txt > normalized_log.txt
```

84. Extract all domain names, convert them to lowercase, sort them, remove duplicates, and save them to unique_domains.txt.

```
grep -Eo '([[:alnum:]]-]+\.)+[[:alpha:]]{2,}' security_log.txt | tr '[:upper:]' '[:lower:]' | sort -u > unique_domains.txt
```

85. Find all failed-login entries and extract only the usernames.

```
grep -i "Failed login" security_log.txt | grep -Eo 'user [[:alnum:]]_+' | awk '{print $2}'
```

86. Find all failed-login entries, extract only the source IP addresses, and display them in sorted order.

```
grep -i "Failed login" security_log.txt | grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' | sort -V
```

87. Extract IP addresses associated with failed logins and determine the unique IPs.

```
grep -i "Failed login" security_log.txt | grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' | sort -V -u
```

88. Extract usernames involved in failed logins, convert them to uppercase, and remove duplicate values.

```
grep -i "Failed login" security_log.txt | grep -Eo 'user [[:alnum:]]_+' | awk '{print $2}' | tr '[:lower:]' '[:upper:]' | sort -u
```

89. Find all lines containing Domain accessed, extract only the domain names, and convert them to lowercase.

```
grep "Domain accessed" security_log.txt | grep -Eo '([[:alnum:]]-]+\.)+[[:alpha:]]{2,}' | tr '[:upper:]' '[:lower:]'
```

261100690009 – Anshul A Salian

90. Find all lines containing MD5, extract only the MD5 hash values, and convert hexadecimal letters to uppercase.

```
grep "MD5=" security_log.txt | grep -Eo 'MD5=[[:xdigit:]]{32}' | cut -d= -f2 | tr '[:lower:]' '[:upper:]'
```

91. Find entries containing User-Agent:, extract the user-agent field, and convert it to lowercase.

```
grep "User-Agent:" security_log.txt | sed 's/.*User-Agent: //' | tr '[:upper:]' '[:lower:]'
```

92. Find all CVE-related lines, extract the CVE identifiers, and convert them to lowercase.

```
grep -Eo 'CVE-[0-9]{4}-[0-9]{4,}' security_log.txt | tr '[:upper:]' '[:lower:]'
```

93. Find all entries containing .exe, extract the relevant filename, and convert it to uppercase.

```
grep -Eo '([[:alnum:]]_\.)+\.exe\b' security_log.txt | tr '[:lower:]' '[:upper:]'
```

94. Extract all domains from Domain accessed entries, convert them to uppercase, sort them, and remove duplicates.

```
grep "Domain accessed" security_log.txt | grep -Eo '([[:alnum:]]+\.)+[[:alpha:]]{2,}' | tr '[:lower:]' '[:upper:]' | sort -u
```